

REST

SYSTEM INTEGRATION



Arturo Mora-Rioja
amri@ek.dk

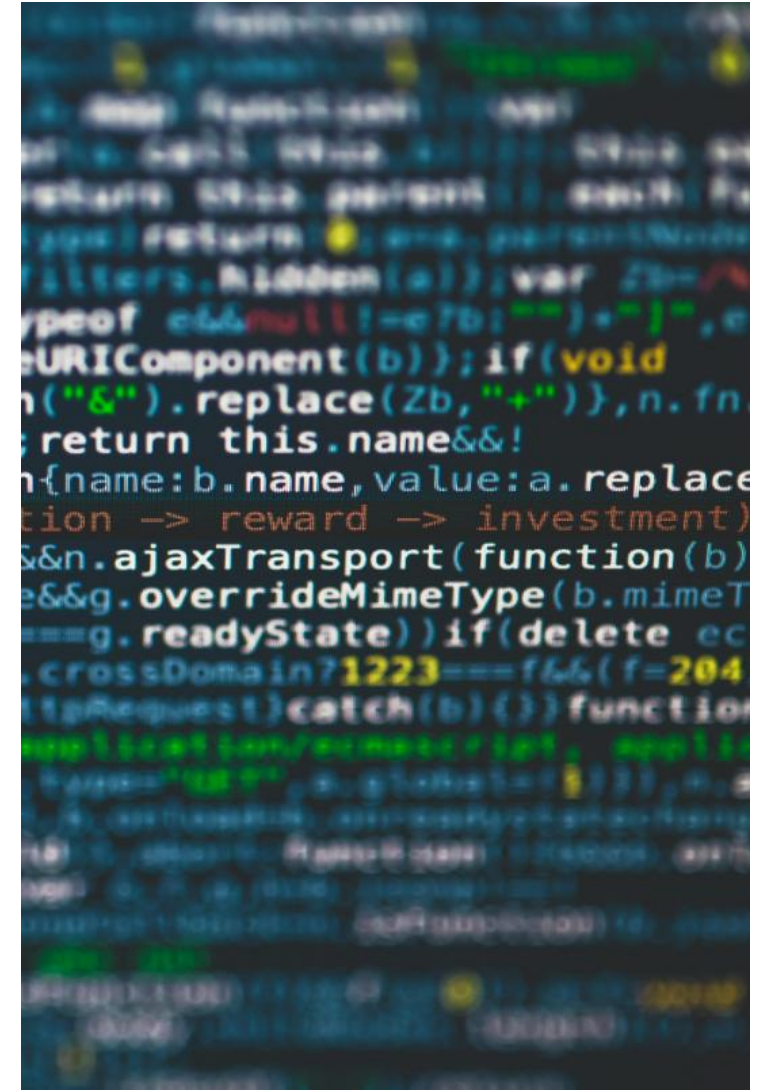
EK

REST – REpresentational State Transfer

- An architectural style, not a protocol
- It treats everything as a **resource** identified by a unique URI (Uniform Resource Identifier)
 - A resource can be
 - A physical entity (e.g., a file)
 - An abstract concept (e.g., a sale, a login)
 - A data entity (e.g., invoices, users, providers)
- Representational
 - It handles the transmission of resources (files in different formats, database rows, code)
 - The client requests a resource on the server
 - The server returns a representation of the resource
- State Transfer
 - The resource's representation is a transfer of its state

REST

- It uses HTTP as transport media
 - Standard use of HTTP verbs
 - Normalised URL structure
- It supports different formats
 - JSON
 - XML
 - HTML
 - Text
 - JavaScript
 - Custom types
 - Binary (e.g., images, pdf files)
- Used for web integration and mobile apps, among others



Standard URL Structure

- Before REST
 - GET `/?id=123&action=delete&type=user`
 - POST `/`
Body = `"delete;type=user;"`
Cookie = `"UserId:123"`
 - DELETE `/?id=123&type=user`
- REST URL
 - DELETE `/users/123`

 - Action (HTTP verb)
 - Resource
 - Resource ID
- Pagination
 - Using query parameters
 - limit. Results per request
 - offset. Result number where to start retrieving results
 - Example
GET `/users?limit=25&offset=51`

REST Naming Convention

- Since REST represents data as resources,
 - Resources are named as nouns in plural `/books`
 - No verbs can be used to name resources
 - ~~`/books/add`~~
 - The HTTP verb specifies the action `POST /books`
- Resource identification takes place in the URL `DELETE /books/132`
- Search, pagination and filtering info as query parameters `GET /books?s=Odyssey`
- Resources may be nested `GET /books/132/authors`
- Compound names must use hyphens rather than underscores `POST /external-loans`
- File extensions must be avoided ~~`/books.php`~~
- REST API Resource Naming Guide: <https://restfulapi.net/resource-naming/>

RESTful APIs

- Roy Fielding [defined](#) in 2000 the six constraints that a fully RESTful API must follow:
 1. **Client-Server Architecture**
The client manages the UI. The server manages the data
 2. **Statelessness**
The server must not remember previous interactions
 3. **Cacheability**
Responses must be marked as cacheable or non-cacheable
 4. **Layered System**
The client must be unaware of whether there are intermediaries between client and server (e.g., CDN, mirror, load balancer, security layer)
 5. **Code on Demand** (optional)
Servers should be allowed to transfer executable code to the client (e.g., JavaScript, plugins)

6. Uniform Interface

a. **Resource Identification in Request**

The URI request must specify what resource it is looking for and what format should be used

b. **Resource Manipulation Through Representations**

A client can modify or delete a resource through its representation

c. **Self-Descriptive Messages**

Each representation must describe its data format

d. **HATEOAS (Hypermedia as the Engine of Application State)**

A client should be able to discover all available resources through the hyperlinks provided

Statelessness

- If the server is aware that it has sent a specific client 10 specific resources and then sends 10 different ones (maybe by keeping track of a session), the server is stateful, thus the API is not RESTful
- However, if client data is stored in a database (thus not handled by the API itself), the API can still be considered stateless
- Client-side session information (e.g., cookies, sessionStorage) is irrelevant to consider the API stateful or stateless
- REST requests must send all the information needed (e.g., pagination related, with an offset parameter and a limit parameter)
- If the backend is made up of different servers, stateless is positive
 - Otherwise, the servers would not be able to keep track of state unless the load balancer relays state information, which is not its job
 - This means that **REST APIs scale horizontally**

HATEOAS

- API self-descriptiveness
 - The API response must include the URIs for the relevant, next-possible actions related to the current resource
- Best practice
 - Prefix metadata by an underscore:

```
{
  "id": 123,
  "name": "Product A",
  "category": "Electronics",
  "_links": [
    {
      "rel": "self",
      "href": "/products/123"
    },
    {
      "rel": "category",
      "href": "/categories/electronics"
    }
  ]
}
```

Based on © 2023 Edet



© [Mike Murray](#) @ Pexels

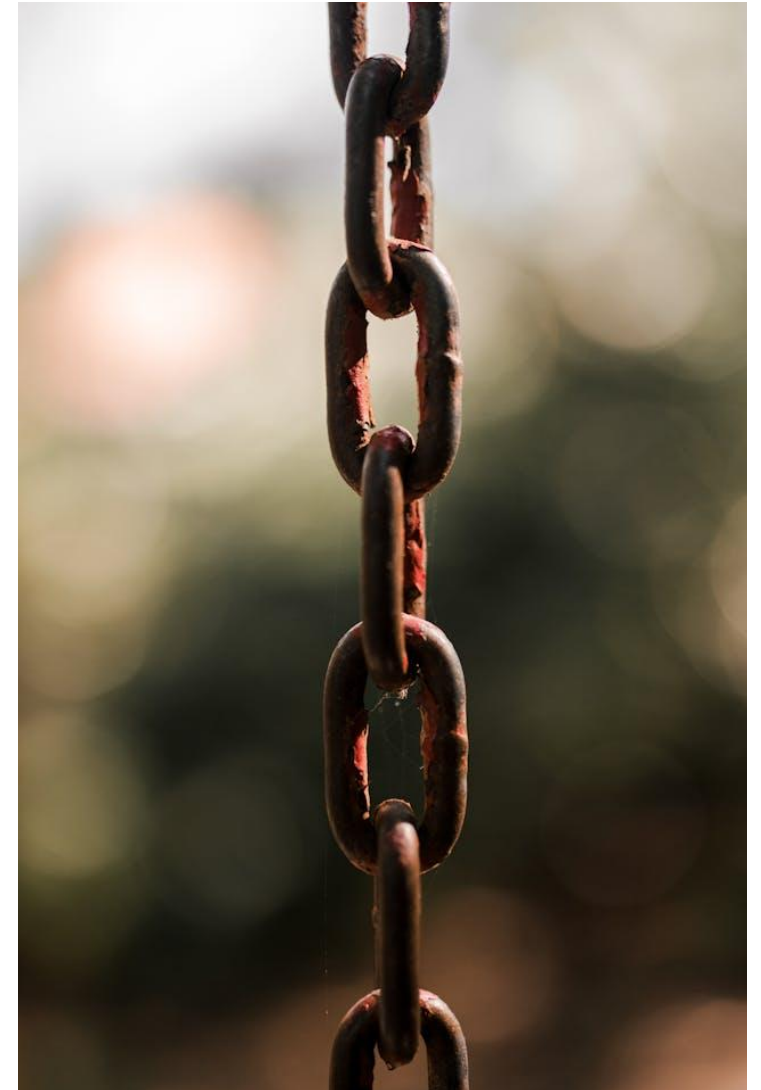
HATEOAS

- Example showcasing contextual links

```
{
  "orderId": 42,
  "status": "pending",
  "_links": {
    "self": { "href": "/orders/42" },
    "cancel": { "href": "/orders/42/cancellations" },
    "items": { "href": "/orders/42/items" }
  }
}
```

- Example showcasing top-level navigational links (optional)

```
{
  "_links": {
    "users": { "href": "/users" },
    "orders": { "href": "/orders" },
    "products": { "href": "/products" }
  }
}
```



© [Lola Russian](#) @ Pexels

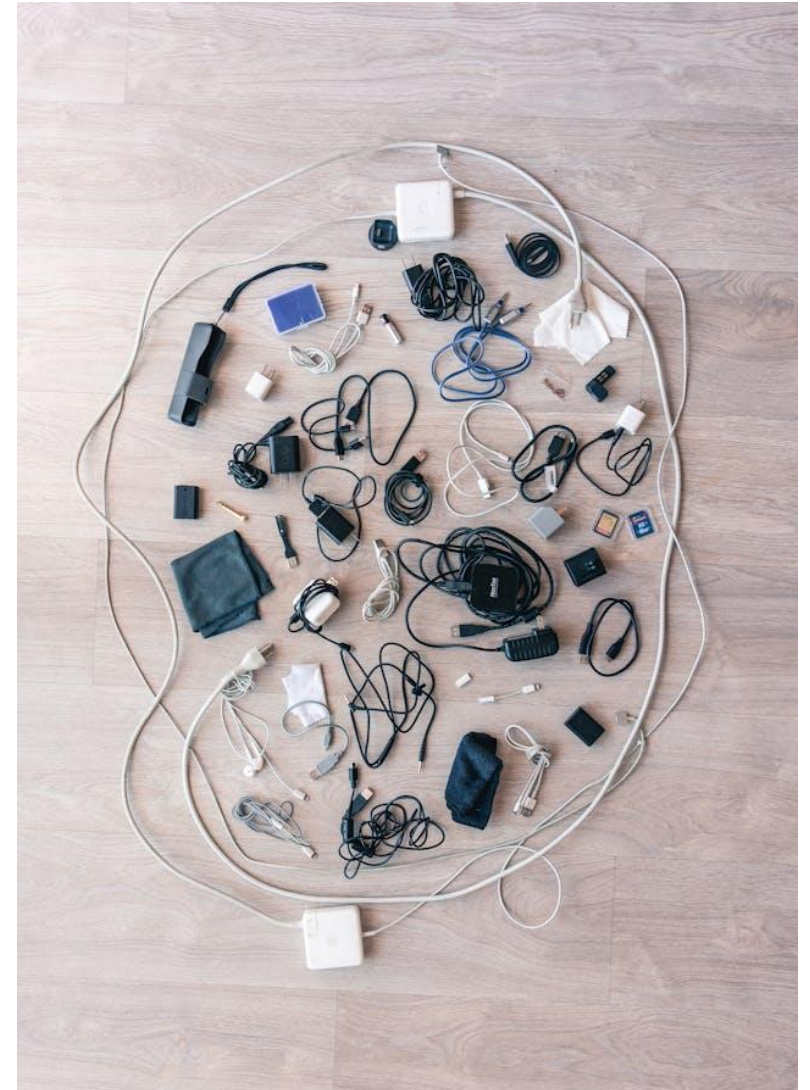
HATEOAS

- 👍
 - Discoverability
 - Loose coupling between client and server
 - Simplified client logic
- 👎
 - Complexity
 - Learning curve



HATEOAS

- There are different conventions, but no clear standard:
 - [HAL](#) – Hypertext Application Language
 - [JSON API](#)
 - [JSON-LD](#) – JSON for Linking Data
 - [Collection+JSON](#)
 - [SIREN](#)
- [On choosing a hypermedia type for your API](#)



© [Hana Brannigan](#) @ Pexels

Versioning Strategies

- **URI Versioning**

- Version information is part of the base URL
- GET /v1/products/123
- 👍
 - Clear and straightforward
 - Enables clients to request a specific version, if available
- 👎
 - Longer and less aesthetically pleasing URIs
 - Increased complexity to handle changes gracefully

Versioning Strategies

- **Header Versioning**

- Version information is included in the HTTP header of the API request
- Either the Accept or Custom header can be used
- GET /products/123
Accept: application/json
X-API-Version: 1
- 
 - Cleaner URIs
 - Elegant
- 
 - Clients are required to include the version header in every request
 - Decreased discoverability

Versioning Strategies

- **Media Type Versioning**

- Different media types (e.g., JSON, XML) can be used for each API version
- The Accept header includes the version alongside the media type
- GET /products/123
Accept: application/vnd.myapi.v1+json
- 👍
 - Cleaner URIs
 - Flexibility in the choice of media types
- 👎
 - Clients are required to include the version header in every request
 - Decreased discoverability

Versioning Strategies

- **Content Negotiation**

- A subtype of media type versioning
- The API serves different versions. The client specifies the desired version in the request
- GET /products/123
Accept: application/vnd.myapi.v2+json
- 👍
 - Cleaner URIs
 - Elegant
- 👎
 - Clients are required to include the version header in every request
 - Decreased discoverability
 - Increased logic handling on the server

Pagination and Filtering

- **Pagination**

- Breaking a large dataset into smaller, discrete portions (pages) that can be retrieved incrementally
- GET `/products?page=2&size=10`
- Best practices
 - Provide links or metadata for navigation to the next and previous pages
 - Include information about the total number of items and pages available

```
{
  "_links": {
    "self": { "href": "/products?page=2&size=10" },
    "next": { "href": "/products?page=3&size=10" },
    "prev": { "href": "/products?page=1&size=10" },
    "first": { "href": "/products?page=1&size=10" },
    "last": { "href": "/products?page=5&size=10" }
  },
  "page": {
    "number": 2,
    "size": 10,
    "totalItems": 42,
    "totalPages": 5
  },
  "_embedded": {
    "products": [
      {
        "id": 11,
        "name": "Wireless Mouse",
        "_links": {
          "self": { "href": "/products/11" }
        }
      },
      /* ...remaining 9 items... */
    ]
  }
}
```



Pagination and Filtering

- **Filtering**

- The dataset is narrowed down based on specific criteria or conditions
- `GET /products?category=electronics&price_max=500`
- Best practices
 - Document available filter parameters and their usage
 - Support complex filtering logic (e.g., AND, OR, ranges)
 - Provide informative error responses for invalid filters and edge cases
- Pagination and filtering can be combined
 - `GET /products?page=2&size=10&category=electronics&price_max=500`

Pagination and Filtering

- **Filtering** (cont.)

- **Example**

```
{
  "_links": {
    "self": { "href": "/products?category=electronics&price_max=500&page=1&size=10" },
    "next": { "href": "/products?category=electronics&price_max=500&page=2&size=10" },
    "first": { "href": "/products?category=electronics&price_max=500&page=1&size=10" },
    "last": { "href": "/products?category=electronics&price_max=500&page=3&size=10" },
    "filters": { "href": "/products/filters" }
  },
  "filters": {
    "current": {
      "category": "electronics",
      "price_max": 500
    },
    "available": {
      "category": {
        "type": "string",
        "allowedValues": ["electronics", "books", "clothing", "toys"],
        "description": "Restricts products to a specific category."
      },
      "price_min": {
        "type": "number",
        "description": "Minimum product price."
      },
      "price_max": {
        "type": "number",
        "description": "Maximum product price."
      }
    },
    ...
  }
}
```

```
...
  "rating_min": {
    "type": "number",
    "description": "Minimum customer rating (1-5)."
  },
},
"syntax": {
  "pattern": "/products?{filter}={value}",
  "multipleFilters": "Combine multiple filters using `&`.",
  "caseSensitivity": "Parameter names are case-sensitive; values depend on the field."
},
},
"page": {
  "number": 1,
  "size": 10,
  "totalItems": 23,
  "totalPages": 3
},
"_embedded": {
  "products": [
    {
      "id": 1011,
      "name": "Noise-Cancelling Headphones",
      "price": 299,
      "category": "electronics",
      "_links": { "self": { "href": "/products/1011" } }
    },
    /* ...9 more items... */
  ]
}
}
```

Sources

- **Edet, Theophilus.** *Web Services & Service-Oriented Architecture.* CompreQuest, 2023.
- **Neetcode.** [*System Design for Begginers.*](#) Neetcode, 2025.
- **Rand-Hendriksen, Morten.** [*Learning REST APIs.*](#) LinkedIn Learning, 2025.
- **Samarakkody, Buddhini.** [*Jakarta Web Services.*](#) LinkedIn Learning, 2023.
- **Soshin, Alexey.** [*Pragmatic System Design.*](#) Udemy, 2025.
- **Williams, Kesha.** [*Programming Foundations: APIs and Web Services.*](#) LinkedIn Learning, 2025.
- Cover picture by [anna-m. w.](#) from [Pexels](#)